Debug Report: What Broke and How I Fixed It

Repository owner

July 2026

Abstract

This is the companion postmortem to the main report. Where that document presents the framework and its measured results, this one is the honest account of the problems I hit building it, grouped by theme, with the symptom, the root cause, and the fix for each. It is a deliverable of equal rank, kept to honest length. The primary source is the dated engineering log kept from the first phase.

Contents

1 Tooling and build surprises

1.1 find_package rejected the version I pinned

The first wall was CMake. I wanted to pin the LLVM major version, so I wrote `find_package(LLVM 21 REQUIRED CONFIG)`. It failed, reporting that the `LLVMConfig.cmake` advertising version 21.1.8 was considered but not accepted. LLVM's generated version config does not treat a bare major of 21 as compatible with the concrete 21.1.8 it ships. Requesting the full patch version would hard code a number that changes on every apt update. The fix was to take whatever configuration `LLVM_DIR` points at and enforce the pin myself with a fatal error guard on `LLVM_VERSION_MAJOR`. The pin stays at major granularity and still fails loudly on the wrong toolchain.

1.2 The plugin linked, the unit tests did not

Linking the standalone unit test executable against the LLVM component libraries surfaced a second issue: `LLVMSupport` declares a link dependency on `zstd::libzstd_shared`, an imported target that did not exist because I had not asked for it. The distribution LLVM was built with zstd. The fix was to resolve the zstd package before LLVM's config is read, with a small shim that constructs the imported target from the library on disk if the system's zstd ships no CMake package, so a continuous integration machine stays portable.

1.3 Spaces in the checkout path broke lit

The plugin built and ran by hand, but the lit smoke test failed with FileCheck complaining of too many positional arguments and a library that could not be loaded. The checkout lives under a Windows Documents folder mounted into WSL, whose path contains spaces. lit substitutes paths as raw text into a RUN line that the shell then splits on those spaces. Rather than move the project, I made quoting a project convention: every RUN line wraps its substitutions in double quotes. This is strictly more robust, since it also survives a user cloning into a spaced path, which LLVM's own in tree tests never have to consider.

1.4 optnone made the whole pipeline a no-op

The most surprising one. The first time I ran the pipeline on a real kernel, it reported zero rewrites on functions I knew were full of identities. `clang -O0` attaches the `optnone` attribute to every function, and the new pass manager honors it by skipping optimization entirely. My passes were loaded and registered; they were simply never run. The fix was one flag, `-Xclang -disable-O0-optnone`, which keeps the naive unoptimized baseline I wanted while letting `opt` actually transform it. The harness sets it for every kernel and records it in the manifest.

2 Pass manager mechanics

2.1 Proving invalidation instead of assuming it

The new pass manager's preservation contract is easy to get subtly wrong. A transform returns a `PreservedAnalyses` set, and it is tempting, or simply habitual, to return `all()` and move on. If the transform actually changed the IR, that leaves later passes reading stale cached analysis results. This is exactly the class of mistake the project is meant to demonstrate command over, so I did not want to trust my own eye on it.

The transforms here change instructions and values but never the control flow graph, so the honest answer is to preserve the CFG set and nothing else. To make sure that stays true, I wrote an invalidation test: it caches an opcode count, runs the algebraic simplifier through a real function pass manager so the invalidation machinery actually runs, then asks for the count again and asserts it dropped. If the simplifier returned `all()`, the manager would hand back the stale count and the test would fail.

I did not just assume the test was meaningful. I temporarily changed the simplifier to return `PreservedAnalyses::all()`, rebuilt, and watched the invalidation test fail; then I reverted and watched it pass. That experiment is what makes the test trustworthy: it fails for the reason it claims to.

2.2 The chain depth that could have looped forever

The use-def chain depth metric walks operand dependencies looking for the longest chain. In straight line code the dependency graph is acyclic, but a phi node in a loop can make a value depend, transitively, on itself, which would send a naive recursion into an infinite loop. I resolved it by treating phi nodes as chain roots: the traversal stops at a phi and does not recurse into its operands. This keeps the walk finite and gives the metric a clean meaning, the longest straight line data dependency, with loop carried dependencies deliberately not counted. A unit test on a loop kernel exists mostly to prove the traversal terminates at all.

3 Pattern correctness

3.1 The differential gate earns its keep

The single most important safety mechanism in the project is the differential execution check. Every kernel prints a checksum of its computation, and the harness aborts unless the baseline, the framework, and `opt -O1` all print the same value. A transform that removes instructions but changes the answer is a miscompile, and without this gate it would look like a great result: fewer instructions, faster runtime, wrong output. With the gate, that outcome is a failed run instead.

This shaped how I wrote the transforms. Every rewrite is value preserving by construction on scalar integers, and the pass refuses to touch anything it cannot prove safe. Floating point is the clearest example: an add of zero is not an identity for floating point without fast math flags, because it changes the sign of a negative zero. The simplifier never even considers it, because its predicates only match integer constants, and a negative test pins that a floating point add of zero survives untouched.

3.2 Negative tests are half the suite

For every positive rewrite there is a negative test proving the unsafe neighbor is left alone: a multiply by a non power of two is not turned into a shift, a volatile load is not eliminated as dead, a store and a call with effects are kept, and two constant operands are not folded because this pass does not claim to do constant folding. Writing these was as valuable as writing the rewrites, because they document the exact boundary of what each pass promises, and they would catch a future change that quietly over reached.

3.3 Strength reduction and the wrap flags

Turning a multiply by a power of two into a shift is only correct if the overflow behavior is preserved. A `mul nsw` promises no signed overflow, and the equivalent shift must carry that promise, so the pass copies the no signed wrap and no unsigned wrap flags onto the shift it creates. A lit test checks the flags appear on the result, not just the shift itself.

4 Measurement noise

The timing results come from a single machine running under WSL2, and I decided early not to pretend otherwise. There is no control over the CPU frequency governor under WSL, so the samples carry a few milliseconds of jitter that no amount of averaging removes. The kernels run in tens of milliseconds, so for the kernels whose structural change is small, the runtime difference genuinely falls inside the measurement spread.

The tempting move was to inflate the workloads until every kernel showed a clean runtime difference. I did not do that, because it would manufacture a runtime story that the passes did not earn. Instead the harness reports the median with the interquartile range, the runtime figure draws that range as an error bar so a reader can see which deltas are signal, and the prose labels the noise dominated kernels as structural only. One kernel, the arithmetic heavy one with real redundancy, shows a runtime improvement clearly outside the spread, and that is the one case where a runtime claim is warranted.

The timing hygiene I could apply, I did: a fixed repetition count with warmup runs discarded, the median rather than the mean so a single slow sample does not dominate, and pinning to a fixed set of cores with `taskset`. The manifest records the repetition count, the core list, and a note that the governor is unavailable, so the conditions travel with the numbers. Better numbers would come from bare metal Linux with a pinned governor, which is noted as future work rather than quietly assumed.

5 Closing

Looking back, the problems fell into a few families. The tooling surprises (the version pin, the zstd target, the spaces in the path, and optnone) were each a short fix once understood, but each cost real time to diagnose because the error message pointed at a symptom rather than the cause. The pass manager mechanics were where the actual learning was: the preservation contract is simple to state and easy to get wrong, and the only way I trusted my implementation was to write a test that fails when it is wrong and prove that it does. The correctness work was mostly discipline, refusing to rewrite anything I could not prove safe, and backing that refusal with negative tests. The measurement work was about honesty under imperfect conditions.

If there is one habit this project reinforced, it is that a guardrail you have watched fail is worth more than one you have only reasoned about. The differential gate, the invalidation test, and the generated figures each exist because I did not want to trust an assumption I could instead check.